Implementing Interactive Theorem Prover from Scratch

HIDEAKI TAKAHASHI, Columbia University, USA

TBD

1 PROPOSITIONS AS TYPES

A theorem prover has to decide whether a proof is correct. A proof written in English cannot be checked by a computer, and the Curry–Howard correspondence is exactly the concept that connects logic with programming: *propositions are types, and proofs are programs.* A proposition A is true if we can construct a term t of type A, written $t : A$.

For example, consider the proposition $A \to B$ ("A implies B"). To prove it, we assume A and construct a proof of B. This is exactly a function that takes a value of type A and returns a value of type B. Thus, logical implication corresponds to a function type. Here are three examples, where $\lambda x.\, e$ denotes a function taking x and returning e:

$$\begin{array}{rcll} P \to P & : & \lambda p.\, p & \text{Return the assumption.} \\ A \to B \to A & : & \lambda a.\, \lambda b.\, a & \text{Ignore the second argument.} \\ (A \to B) \to (B \to C) \to A \to C & : & \lambda f.\, \lambda g.\, \lambda a.\, g(f(a)) & \text{Compose two functions.} \end{array}$$

The same idea extends to universal quantification. To prove "for every $x : A$, $B(x)$ holds", we construct a function that takes any $x : A$ and returns a proof of $B(x)$. Unlike an ordinary function type $A \to B$, the return type $B(x)$ may depend on the input x. This is called a *dependent function type*, written

$$\Pi(x : A).\, B(x).$$

For example, "every proposition implies itself" is expressed as

$$\Pi(P{:}\text{Sort } 0).\, P \to P,$$

where Sort 0 is the type of propositions. Its proof is simply

$$\lambda(P{:}\text{Sort } 0).\, \lambda(p{:}P).\, p.$$

This gives us the central idea behind a theorem prover: *proving a theorem means constructing a term of the required type, and verifying a proof means type-checking that term.* The kernel implements these typing rules rather than reasoning about truth directly.

2 SYNTAX

We begin with a small dependent type theory consisting of five kinds of terms:

$$t, A, B ::= \text{Sort } u \mid x \mid \Pi(x{:}A).\, B \mid \lambda(x{:}A).\, t \mid t\ s$$

Here, u is a natural number and x is a variable name. Throughout, $s, t, a, b, f, A, B, C, T$ (possibly with subscripts or primes) stand for terms, x, y, z and the other names bound by Π or λ stand for variable names, and u, v stand for universe levels. Accordingly, the set $\text{FV}(t)$ of free variables of a term t, defined in §3, is a set of names.

- Sort u: A universe containing types.
- x: A variable.
- $\Pi(x{:}A).\, B$: A Π-*type* (dependent function type), whose result type B may depend on x.
- $\lambda(x{:}A).\, t$: A λ-*abstraction*, i.e. a function that takes $x : A$ and returns t.

Author's address: Hideaki Takahashi, ht2673@columbia.edu, Columbia University, USA.

- $t\ s$: Function application.

When B does not depend on x, we write $A \to B$ instead of $\Pi(x{:}A).\,B$. Thus, ordinary function types are special cases of dependent function types. For example, the proposition "every proposition implies itself" and its proof are represented as

$$\underbrace{\lambda(P{:}\mathrm{Sort}\ 0).\,\lambda(p{:}P).\,p}_{\text{proof}} \quad : \quad \underbrace{\Pi(P{:}\mathrm{Sort}\ 0).\,P \to P}_{\text{proposition = type of proof}}.$$

Notice that types and expressions share the same grammar. There is no separate syntax for types: whether a term is a valid type is determined by the type checker.

Application associates to the left and arrows associate to the right: $f\ a\ b$ means $(f\ a)\ b$, and $A \to B \to C$ means $A \to (B \to C)$. A function of two arguments is thus a function that takes the first argument and returns a function expecting the second; this encoding is called *currying*. For example, $\lambda(a{:}A).\,\lambda(b{:}B).\,a$ has type $A \to B \to A$, and applied to $x : A$ and $y : B$ it is written $(\lambda(a{:}A).\,\lambda(b{:}B).\,a)\ x\ y$, that is, $((\lambda(a{:}A).\,\lambda(b{:}B).\,a)\ x)\ y$.

3 FREE VARIABLES AND SUBSTITUTION

Π and λ *bind* their variable: in $\lambda(x{:}A).\,t$, occurrences of x inside t refer to the argument. The binder's scope is the body only; x is not bound in the annotation A.

Definition 1 (Free variables).

$$\begin{aligned}
\mathrm{FV}(\mathrm{Sort}\ u) &= \emptyset & \mathrm{FV}(\Pi(x{:}A).\,B) &= \mathrm{FV}(A) \cup (\mathrm{FV}(B) \setminus \{x\}) \\
\mathrm{FV}(x) &= \{x\} & \mathrm{FV}(\lambda(x{:}A).\,t) &= \mathrm{FV}(A) \cup (\mathrm{FV}(t) \setminus \{x\}) \\
\mathrm{FV}(t\ s) &= \mathrm{FV}(t) \cup \mathrm{FV}(s)
\end{aligned}$$

The typing rule for application (§6) needs to replace a variable with a term. We write $t[x \mapsto s]$ for the result of replacing the free occurrences of x in t with s.

The only subtle case is a binder. Consider $(\lambda(y{:}A).\,x)[x \mapsto y]$. The body says "ignore the argument and return the outer x". Replacing x by y naively gives $\lambda(y{:}A).\,y$, the identity function, which means something else: the free y has been *captured* by the binder. Substitution must avoid this by renaming the binder first.

Definition 2 (Capture-avoiding substitution). *For $\square \in \{\Pi, \lambda\}$:*

$$\begin{aligned}
x[x \mapsto s] &= s \\
y[x \mapsto s] &= y && (y \neq x) \\
\mathrm{Sort}\ u[x \mapsto s] &= \mathrm{Sort}\ u \\
(t_1\ t_2)[x \mapsto s] &= t_1[x \mapsto s]\ \ t_2[x \mapsto s] \\
(\square(x{:}A).\,B)[x \mapsto s] &= \square(x{:}A[x \mapsto s]).\,B && \textit{(we say that } x \textit{ is shadowed)} \\
(\square(y{:}A).\,B)[x \mapsto s] &= \square(y{:}A[x \mapsto s]).\,B[x \mapsto s] && (y \neq x,\ y \notin \mathrm{FV}(s)) \\
(\square(y{:}A).\,B)[x \mapsto s] &= \square(z{:}A[x \mapsto s]).\,(B[y \mapsto z])[x \mapsto s] && (y \neq x,\ y \in \mathrm{FV}(s),\ z \textit{ fresh})
\end{aligned}$$

Here "z fresh" means $z \notin \mathrm{FV}(s) \cup \mathrm{FV}(B) \cup \{x\}$.

4 ALPHA-EQUIVALENCE

The names of bound variables carry no meaning. For example, $\lambda(x{:}A).\,x$ and $\lambda(y{:}A).\,y$ are the same function, and $\Pi(P{:}\mathrm{Sort}\ 0).\,P \to P$ and $\Pi(Q{:}\mathrm{Sort}\ 0).\,Q \to Q$ are the same proposition. Terms that differ only in the names of bound variables are called *α-equivalent*.

Definition 3 (α-equivalence). *$=_\alpha$ is the smallest congruence on terms containing, for $\square \in \{\Pi, \lambda\}$,*

$$\square(x{:}A).\, B \;=_\alpha\; \square(y{:}A).\, B[x \mapsto y] \qquad \text{whenever } y \notin \mathrm{FV}(B).$$

This definition is easy to state but not directly executable, since it does not say which names to choose. An implementation instead walks both terms in lockstep and remembers, for each side, the list of binders passed so far. A bound variable is then identified by *which* binder it refers to, not by its name.

To describe this walk, we use a judgment of the form $E_1; E_2 \vdash s =_\alpha t$, where E_1 and E_2 are lists of names. The list E_1 records the binders that enclose s, from the outermost to the innermost, and E_2 records those that enclose t. The judgment reads: "s, placed inside the binders E_1, and t, placed inside the binders E_2, are the same term up to the names of bound variables". Two lists are needed because the two terms may use different names for corresponding binders. For example, comparing $\lambda(x{:}A).\, \lambda(y{:}B).\, x$ with $\lambda(a{:}A).\, \lambda(b{:}B).\, a$ passes two binders on each side and arrives at

$$x, y;\; a, b \vdash x =_\alpha a,$$

which holds because x and a both refer to the outer of the two binders. Comparing two whole terms starts from empty lists, $\cdot\,;\cdot \vdash s =_\alpha t$. The same symbol $=_\alpha$ appears in Definition 3 and in the judgment; the prefix $E_1; E_2 \vdash$ distinguishes the judgment, and Theorem 1 below shows that the two agree.

To say "the same binder" precisely, let $\mathit{idx}(E, x)$ be the position of the last occurrence of x in E, counted from the right starting at 0, or undefined if $x \notin E$. In the example, $\mathit{idx}(x, y;\, x) = \mathit{idx}(a, b;\, a) = 1$. Taking the *last* occurrence handles shadowing: in $\lambda(x{:}A).\, \lambda(x{:}B).\, x$ the body refers to the inner x.

$$\frac{}{E_1; E_2 \vdash \mathrm{Sort}\ u =_\alpha \mathrm{Sort}\ u}\ (\alpha\text{-Sort}) \qquad \frac{\mathit{idx}(E_1, x) = \mathit{idx}(E_2, y) = i}{E_1; E_2 \vdash x =_\alpha y}\ (\alpha\text{-Bound}) \qquad \frac{x \notin E_1 \qquad x \notin E_2}{E_1; E_2 \vdash x =_\alpha x}\ (\alpha\text{-Free})$$

$$\frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash B =_\alpha B'}{E_1; E_2 \vdash \Pi(x{:}A).\, B =_\alpha \Pi(y{:}A').\, B'}\ (\alpha\text{-Pi}) \qquad \frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash t =_\alpha t'}{E_1; E_2 \vdash \lambda(x{:}A).\, t =_\alpha \lambda(y{:}A').\, t'}\ (\alpha\text{-Lam})$$

$$\frac{E_1; E_2 \vdash t_1 =_\alpha t_1' \qquad E_1; E_2 \vdash t_2 =_\alpha t_2'}{E_1; E_2 \vdash t_1\ t_2 =_\alpha t_1'\ t_2'}\ (\alpha\text{-App})$$

Fig. 1. Algorithmic α-equivalence. By Theorem 1, two terms s and t are α-equivalent iff $\cdot\,;\cdot \vdash s =_\alpha t$.

Free variables are compared by name (α-Free), and bound variables by position (α-Bound). A variable that is bound on one side and free on the other is never equal to anything, which is why there is no rule for that case. Because both sides descend through binders together, E_1 and E_2 always have the same length. So comparing positions counted from the right is the same as comparing positions counted from the left, and an implementation may store each list as a map from names to binding depths, provided it restores the previous entry when it leaves a binder. Using positions instead of names everywhere gives the *de Bruijn representation* [6], in which α-equivalent terms are syntactically identical.

The rules of Fig. 1 are a different definition from Definition 3, so we must check that they agree.

Theorem 1 (Correctness of algorithmic α-equivalence). *For all terms s and t, $s =_\alpha t$ in the sense of Definition 3 if and only if $\cdot\,;\cdot \vdash s =_\alpha t$ is derivable by the rules of Fig. 1.*

The idea of the proof is that both sides say the same thing: s and t have the same de Bruijn representation. For the algorithm this is almost immediate, since its rules compare positions exactly as de Bruijn indices do. For Definition 3, the key fact is that renaming a bound variable does not change the de Bruijn representation. Appendix A gives the full proof.

5 REDUCTION AND CONVERSION

Applying a λ-abstraction to an argument is computed by substituting the argument for the bound variable:

$$(\lambda(x{:}A).\, t)\ s\ \ \longrightarrow_\beta\ \ t[x \mapsto s].$$

This step is called *β-reduction*. A subterm of the form $(\lambda(x{:}A).\, t)\ s$ is called a *β-redex*, and we also allow this step anywhere inside a term. A term with no β-redex is in *normal form*.

Computation can make two different-looking types equal. If $Id = \lambda(X{:}\mathrm{Sort}\ 0).\, X$, then a proof of $Id\ P$ should also be a proof of P, since $Id\ P \longrightarrow_\beta P$. *Definitional equality* (or *conversion*) makes such types interchangeable by identifying terms that compute to the same result.

Definition 4 (Conversion). *$s \equiv t$ is the smallest equivalence relation that contains $=_\alpha$ and $\longrightarrow_\beta$ and is closed under all term constructors.*

To decide $s \equiv t$, the kernel normalizes both terms and compares the results with $=_\alpha$. This procedure relies on two facts. First, β-reduction is confluent: different reduction orders cannot lead to different normal forms, so $s \equiv t$ holds exactly when the normal forms of s and t are α-equivalent. Second, every well-typed term *has* a normal form, so normalization terminates. The second fact holds for our system, but it is a deep theorem (strong normalization) [5]. A practical kernel also caps the number of reduction steps, so that a bug cannot make it loop forever. Algorithm 1 in §8 includes Normalize as pseudocode.

6 TYPING RULES

Contexts and judgments. A *context* Γ is an ordered list of assumptions $x_1 : A_1, \ldots, x_n : A_n$. It records the type of each variable in scope, and we write $\mathrm{dom}(\Gamma)$ for the set $\{x_1, \ldots, x_n\}$. Since C-Ext below forbids repeated names, Γ can be used like a dictionary from names to types, and T-Var is simply a lookup. Unlike a plain dictionary, however, the order matters: A_i may mention $x_1, \ldots, x_{i-1}$, as in $P : \mathrm{Sort}\ 0, h : P$. We use two judgments:

- $\vdash \Gamma$ ok: the context Γ is well formed.
- $\Gamma \vdash t : A$: under the assumptions in Γ, the term t has type A.

Read through Curry–Howard, $\Gamma \vdash t : A$ says that t is a proof of A from the hypotheses Γ.

We read each rule from the bottom up, as a recipe for checking a term.

C-Empty, C-Ext. A context is built one assumption at a time, and each new assumption must have a type as its type. The side condition $x \notin \mathrm{dom}(\Gamma)$ forbids shadowing inside a context. Since we work up to α-equivalence, this is no restriction on terms: a binder that would shadow can always be renamed first. §10 shows why this condition matters.

T-Sort. Every universe lives in the next one; see §7 for why.

T-Var. A variable has the type it was assumed to have. Logically, this is "use a hypothesis".

T-Pi. $\Pi(x{:}A).\, B$ is a type if A is a type and B is a type once we assume $x : A$. It lives in the larger of the two universes, except that it is a proposition whenever B is one; $\mathrm{imax}(u, v)$ is 0 if $v = 0$ and $\max(u, v)$ otherwise (§7.4). Logically, this rule says when $A \to B$ and $\forall x{:}A.\, B$ are well-formed propositions.

$$\frac{}{\vdash \cdot \text{ ok}}\ (\text{C-Empty}) \qquad \frac{\vdash \Gamma \text{ ok} \qquad \Gamma \vdash A : \text{Sort } u \qquad x \notin \text{dom}(\Gamma)}{\vdash \Gamma, x : A \text{ ok}}\ (\text{C-Ext})$$

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)}\ (\text{T-Sort}) \qquad \frac{\vdash \Gamma \text{ ok} \qquad (x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\text{T-Var})$$

$$\frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Sort imax}(u, v)}\ (\text{T-Pi}) \qquad \frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ (\text{T-Lam})$$

$$\frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x \mapsto a]}\ (\text{T-App}) \qquad \frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : \text{Sort } u \qquad A \equiv B}{\Gamma \vdash t : B}\ (\text{T-Conv})$$

Fig. 2. Well-formed contexts and typing rules.

T-Lam. To build a function from A to B, assume $x : A$ and build $t : B$. Logically, this is the introduction rule for $\to$ and $\forall$: to prove $A \to B$, assume A and prove B. Note that B may mention x, and it is kept as is in the resulting Π type.

T-App. If f maps each $x : A$ to an element of B, and $a : A$, then $f\ a$ has type B with x replaced by a. Logically, this is modus ponens when B does not mention x, and instantiation of a universal statement when it does. For example, if $f : \Pi(n{:}\text{Nat}).\, n + 0 = n$, then $f\ 3 : 3 + 0 = 3$ (natural numbers are added in §12).

T-Conv. Types that are equal up to computation are interchangeable. This is the only rule that is not determined by the shape of t.

Why application matters. Without T-App, a hypothesis can be returned but never used. Roughly, the provable propositions are then those whose conclusion is literally one of the hypotheses, such as $P \to Q \to P$. Composition, $(P \to Q) \to (Q \to R) \to P \to R$, has the proof $\lambda(f{:}P \to Q).\, \lambda(g{:}Q \to R).\, \lambda(p{:}P).\, g\ (f\ p)$, which needs application.

7 UNIVERSES: WHY Sort u?

The typing rules of §6 are meant to be rules of logic. This section first makes that claim precise (§7.1). It then shows that the simplest choice of universes, a single universe that contains itself, makes the system inconsistent (§7.3), and that the hierarchy Sort 0 : Sort 1 : $\cdots$ with an impredicative bottom universe, as in Lean, avoids the problem (§7.4).

7.1 The Typing Rules as a Logic

Consider the logic whose formulas are built from proposition variables $P, Q, \ldots$ by implication and quantification over propositions:

$$\varphi, \psi ::= P \mid \varphi \to \psi \mid \forall P.\, \varphi$$

This is second-order propositional logic. Its proofs are natural-deduction derivations of judgments $\Delta \vdash \varphi$, where Δ is a list of hypotheses. Each formula translates to a type, and each list of hypotheses to a context:

$$P^* = P, \qquad (\varphi \to \psi)^* = \varphi^* \to \psi^*, \qquad (\forall P.\, \varphi)^* = \Pi(P{:}\text{Sort } 0).\, \varphi^*,$$
$$(\varphi_1, \ldots, \varphi_n)^* = P_1 : \text{Sort } 0, \ldots, P_k : \text{Sort } 0,\ h_1 : \varphi_1^*, \ldots, h_n : \varphi_n^*,$$

where $P_1, \ldots, P_k$ are the proposition variables free in Δ and $h_1, \ldots, h_n$ are fresh names for the hypotheses. Fig. 3 places each logical rule next to the typing rule it corresponds to. Erasing the terms from the right column gives the left column exactly.

	Logic	**Type theory**
Hypothesis	$\dfrac{\varphi_i \in \Delta}{\Delta \vdash \varphi_i}$	$\dfrac{(h_i : \varphi_i^*) \in \Delta^*}{\Delta^* \vdash h_i : \varphi_i^*}$ (T-Var)
$\to$-intro	$\dfrac{\Delta, \varphi \vdash \psi}{\Delta \vdash \varphi \to \psi}$	$\dfrac{\Delta^*, h : \varphi^* \vdash t : \psi^*}{\Delta^* \vdash \lambda(h{:}\varphi^*).\, t : \varphi^* \to \psi^*}$ (T-Lam)
$\to$-elim	$\dfrac{\Delta \vdash \varphi \to \psi \qquad \Delta \vdash \varphi}{\Delta \vdash \psi}$	$\dfrac{\Delta^* \vdash f : \varphi^* \to \psi^* \qquad \Delta^* \vdash a : \varphi^*}{\Delta^* \vdash f\ a : \psi^*}$ (T-App)
$\forall$-intro	$\dfrac{\Delta \vdash \varphi \qquad P \notin \mathrm{FV}(\Delta)}{\Delta \vdash \forall P.\, \varphi}$	$\dfrac{\Delta^*, P : \mathrm{Sort}\ 0 \vdash t : \varphi^*}{\Delta^* \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, t : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^*}$ (T-Lam)
$\forall$-elim	$\dfrac{\Delta \vdash \forall P.\, \varphi}{\Delta \vdash \varphi[P \mapsto \psi]}$	$\dfrac{\Delta^* \vdash f : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^* \qquad \Delta^* \vdash \psi^* : \mathrm{Sort}\ 0}{\Delta^* \vdash f\ \psi^* : \varphi^*[P \mapsto \psi^*]}$ (T-App)

Fig. 3. Natural deduction (left) and the typing rules that implement it (right). Premises that only check that a type is well formed are omitted.

In the $\forall$-intro row, the side condition $P \notin \mathrm{FV}(\Delta)$ guarantees that no hypothesis mentions P, so the typing rule may add $P : \mathrm{Sort}\ 0$ after the hypotheses. In the $\forall$-elim row, the premise $\psi^* : \mathrm{Sort}\ 0$ holds for every formula ψ, even one that contains $\forall$, because T-Pi places every translated formula in Sort 0 (§7.4). Hence $\forall$-elim may instantiate P with any formula, as in the logic.

Theorem 2 (Proofs are terms). *If $\Delta \vdash \varphi$ is provable, then $\Delta^* \vdash t : \varphi^*$ for some term t.*

The figure turns each step of a proof into a step of a typing derivation, so every proof becomes a term. Appendix B states the theorem precisely and gives the full proof.

The theorem says that the type checker can check every proof of the logic. For the type checker to be trustworthy, we also need the opposite guarantee: it must not accept a term for a proposition that the logic refutes. Define $\bot = \forall P.\, P$ ("everything holds"). No proof of $\bot$ exists: if each P is read as true or false, every rule preserves truth under every reading, but $\forall P.\, P$ is false. The type theory should agree, that is, no closed term should have type

$$\bot^* = \Pi(P{:}\mathrm{Sort}\ 0).\, P.$$

A closed term $t : \bot^*$ would prove every proposition, since $t\ C : C$ for every $C : \mathrm{Sort}\ 0$. A system in which no closed term has type $\bot^*$ is called *consistent*, and a design for universes is acceptable only if the system remains consistent.

One might hope for the full converse of Theorem 2: every term of type φ^* comes from a proof of φ. That is a stronger property, called conservativity, and it is harder to establish here, because Sort 0 also contains types that are not translations of formulas, such as $\Pi(F{:}\mathrm{Sort}\ 0 \to \mathrm{Sort}\ 0).\, F\ P \to F\ P$, and a term may pass through them. We do not need it; consistency is enough.

7.2 What Is the Type of Sort 0?

The $\forall$ rows of Fig. 3 bind P with the annotation Sort 0. T-Lam and T-Pi require every annotation to be a type, so Sort 0 itself needs a type. Without one, a proposition variable could only be an assumption placed in the context from outside:

$$P : \mathrm{Sort}\ 0 \vdash \lambda(h{:}P).\, h : P \to P \qquad \text{versus} \qquad \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P.$$

The left judgment proves $P \longrightarrow P$ for one fixed P. The right one is a single closed term that proves it for every P, and it can be applied later to any particular proposition. §7.3 shows that the simplest answer, that Sort 0 is its own type, makes the system inconsistent. §7.4 then presents Lean's answer, a hierarchy of universes.

7.3 Type Is Not a Type

Suppose we replace all universes by a single universe Type that contains itself:

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Type} : \text{Type}} \qquad \frac{\Gamma \vdash A : \text{Type} \qquad \Gamma, x : A \vdash B : \text{Type}}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Type}}$$

Translate $\forall P.\, \varphi$ as $\Pi(P{:}\text{Type}).\, \varphi^*$. Then P ranges over a collection that contains Type itself and every type built from it. Such a collection is the type-theoretic analogue of the set of all sets, and naive set theory shows that such collections lead to contradictions. The best-known example is *Russell's paradox*. Let $R = \{\, x \mid x \notin x \,\}$ be the set of all sets that are not members of themselves. If $R \in R$, then R fails the condition, so $R \notin R$; if $R \notin R$, then R satisfies the condition, so $R \in R$. Either way we reach a contradiction, and the culprit is a collection that may contain itself. Type theory has no membership test like $x \notin x$, so Russell's argument does not carry over word for word. Girard instead adapted a related paradox about the collection of all well-orderings, due to Burali-Forti, and showed that the system has a closed term

$$\vdash t_{\mathrm{G}} : \Pi(P{:}\text{Type}).\, P,$$

and Hurkens later found a short one [7, 8]. So the system is inconsistent: $\bot$ has no proof, yet $\bot^*$ has a term, and a kernel built on these rules accepts a proof of every statement.

The damage reaches the algorithm as well. Subject reduction still holds, so if t_{G} had a normal form, it would be a normal closed term of type $\Pi(P{:}\text{Type}).\, P$. The argument at the end of §11 shows that no such normal term exists. Hence t_{G} has no normal form, strong normalization fails, and Normalize in Algorithm 1 can run forever.

7.4 A Hierarchy with an Impredicative Bottom

The second answer is that Sort 0 lives in a larger universe, which lives in a larger one still:

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)}\ (\text{T-Sort}) \qquad \text{so} \qquad \text{Sort } 0 : \text{Sort } 1 : \text{Sort } 2 : \cdots .$$

No universe contains itself. A Π type takes an argument from one universe and returns a type in another, so T-Pi must also say which universe the Π type itself belongs to. Lean's answer uses the function imax:

$$\frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Sort } \operatorname{imax}(u, v)}\ (\text{T-Pi}) \qquad \operatorname{imax}(u, v) = \begin{cases} 0 & \text{if } v = 0, \\ \max(u, v) & \text{otherwise.} \end{cases}$$

The case split in imax makes a Π type impredicative when its body is a proposition, and predicative otherwise.

Propositions are impredicative. If the body B is a proposition ($v = 0$), the whole Π is a proposition, whatever it quantifies over. In particular $(\forall P.\, \varphi)^* = \Pi(P{:}\text{Sort } 0).\, \varphi^*$ is in Sort 0, although it quantifies over all of Sort 0. A universe that contains types quantifying over itself is called *impredicative*. Because every translated formula lies in Sort 0, the premise $\psi^* : \text{Sort } 0$ of $\forall$-elim always holds, so a

universal statement can be instantiated with any proposition, even with itself. For example, with $id = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, h$,

$$id : \Pi(P{:}\text{Sort } 0).\, P \to P \qquad \text{and} \qquad \Pi(P{:}\text{Sort } 0).\, P \to P : \text{Sort } 0,$$

so $id\ (\Pi(P{:}\text{Sort } 0).\, P \to P)\ id$ is well typed, with type $\Pi(P{:}\text{Sort } 0).\, P \to P$. The same case makes $\forall n : \text{Nat},\ n + 0 = n$ a proposition once natural numbers are added in §12, even though Nat lives in Sort 1.

Types are predicative. If the body is not a proposition ($v \geq 1$), the Π lives in the larger of the two universes. A type that quantifies over Sort u then lives in Sort $(\max(u + 1, v))$, which is never Sort u:

$$\frac{\dfrac{}{\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)}\ \textsc{T-Sort} \qquad \Gamma, x : \text{Sort } u \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x{:}\text{Sort } u).\, B : \text{Sort } (\max(u+1, v))}\ \textsc{T-Pi}$$

This property is called *predicativity*. For example, the identity on types $\lambda(A{:}\text{Sort } 1).\, \lambda(a{:}A).\, a$ has type $\Pi(A{:}\text{Sort } 1).\, A \to A$, which lives in Sort 2, so it cannot be applied to its own type; a copy at Sort 2 can. Each level holds the level beneath it, and in practice a few levels suffice; higher levels appear when one quantifies over large collections, such as the category of all small categories.

Why this is safe. Impredicativity looks like the self-reference that §7.3 warned against, but it is confined: Sort 0 contains types that quantify over Sort 0, yet Sort 0 is not a member of itself. Coquand showed that one impredicative universe at the bottom is consistent [2], whereas Girard's paradox can already be derived once two levels are impredicative [8]. With these rules the system is consistent and strongly normalizing (§11).

7.5 Comparison with Lean

In Lean [9], Sort 0 is Prop, Sort $(u + 1)$ is Type u, and Π types are assigned universes with imax exactly as above. Two differences remain. First, Prop in Lean has *definitional proof irrelevance*: any two proofs of the same proposition are definitionally equal. Our conversion does not identify proofs; adding this would make conversion depend on types, so the simple "normalize and compare" check of §5 would no longer suffice. Second, Lean has *universe polymorphism*, so definitions can be stated once for all levels u, and imax then matters for level variables as well. In our system every level is a concrete number.

Table 1. Design choices for universes.

Design	Quantifies over	Drawback
No universes	values only	cannot say "for every proposition"
Type : Type	everything	inconsistent
Two sorts $* : \Box$	$*$ only	$\Box$ itself has no type
Sort u : Sort $(u + 1)$ with max	any level	$\forall$ over propositions is not a proposition
Sort u : Sort $(u + 1)$ with imax (Lean)	any level	normalization is harder to prove

8 A TYPE CHECKING ALGORITHM

The rules in Fig. 2 define which judgments hold, but they are not yet an algorithm, because T-Conv can be applied at any point. The standard solution splits type checking into two mutually recursive functions [5]: Infer computes the type of a term, and Check verifies a term against an expected

type. Conversion is used only where two types must be compared: in Check, and when a type must have a particular shape (a Π or a Sort).

Algorithm 1 Type inference, checking, and normalization.

```
1:  function Infer(Γ, t)
2:      if t = Sort u then return Sort (u + 1)
3:      else if t = x then
4:          if (x : A) ∈ Γ then return A
5:          else fail "unbound variable"
6:      else if t = Π(x:A). B then
7:          u ← InferSort(Γ, A)
8:          (x′, B′) ← Freshen(Γ, x, B)
9:          v ← InferSort((Γ, x′ : A), B′)
10:         return Sort imax(u, v)
11:     else if t = λ(x:A). s then
12:         InferSort(Γ, A)
13:         (x′, s′) ← Freshen(Γ, x, s)
14:         B ← Infer((Γ, x′ : A), s′)
15:         return Π(x′:A). B
16:     else if t = f a then
17:         T ← Normalize(Infer(Γ, f))
18:         if T = Π(x:A). B then
19:             Check(Γ, a, A)
20:             return B[x ↦ a]
21:         else fail "not a function"

22: function Check(Γ, t, T)
23:     T′ ← Infer(Γ, t)
24:     if Normalize(T′) =α Normalize(T) then return
25:     else fail "type mismatch"

26: function InferSort(Γ, A)
27:     T ← Normalize(Infer(Γ, A))
28:     if T = Sort u then return u
29:     else fail "expected a type"

30: function Freshen(Γ, x, b)
31:     if x ∈ dom(Γ) then
32:         choose x′ ∉ dom(Γ) ∪ FV(b)
33:         return (x′, b[x ↦ x′])
34:     else return (x, b)

35: function Whnf(t)
36:     loop
37:         write t as a spine h a_1 ··· a_n
38:         if h = λ(x:A). b and n ≥ 1 then                    ▷ β
39:             t ← (b[x ↦ a_1]) a_2 ··· a_n
40:         else return t

41: function Normalize(t)
42:     t ← Whnf(t)
43:     if t = Π(x:A). B then return Π(x:Normalize(A)). Normalize(B)
44:     else if t = λ(x:A). b then return λ(x:Normalize(A)). Normalize(b)
45:     else write t as a spine h a_1 ··· a_n and return h Normalize(a_1) ··· Normalize(a_n)
```

Algorithm 1 has the following properties.

- Each case corresponds to exactly one rule of Fig. 2, so the algorithm is *syntax-directed*. Finding a proof may be hard, but checking a given proof is mechanical.
- InferSort implements the premises of the form $\Gamma \vdash A : \mathrm{Sort}\ u$. It is also how the kernel decides whether a term is a type at all: A is a type exactly when its type normalizes to a universe. A λ-abstraction can never be a type, because its inferred type is always a Π.
- Freshen renames a binder whose name is already in the context, implementing the side condition of C-Ext.
- Starting from the empty context, every context the algorithm builds is well formed, because each assumption is added only after InferSort succeeds. A caller that starts from a non-empty context must check that context separately.
- Termination depends on Normalize terminating, which holds for well-typed terms by strong normalization (§5).

The last two functions of Algorithm 1 define Normalize. It is built from Whnf, which reduces only the head of a term until no rule applies there, giving its *weak head normal form*. A term is viewed as a *spine* $h\ a_1 \cdots a_n$: a head h that is not an application, applied to $n \geq 0$ arguments. Since application associates to the left (§2), $f\ a\ b\ c$ is the nested term $((f\ a)\ b)\ c$; following the function side of each application down to f gives the head, and the arguments met on the way, read from the inside out, are a, b, c. Every term has exactly one such decomposition; a term that is not an application is a spine with $n = 0$. In the system so far the only reduction is β, so Whnf has a single case; §12.5 adds two more.

Whnf alone is enough when a caller only needs the outermost shape of a type, such as a Π in the application case of Infer. Practical kernels use Whnf there and compare types lazily instead of normalizing them fully; Algorithm 1 normalizes fully for simplicity. Termination of the loop for well-typed terms is strong normalization.

9 WORKED EXAMPLES

This section runs Algorithm 1 on three terms and relates each run to the typing rules of Fig. 2.

The identity proof. Let $\Gamma_1 = P : \mathrm{Sort}\ 0$ and $\Gamma_2 = \Gamma_1, h : P$. Omitting the well-formedness premises $\vdash \Gamma$ ok, the derivation of the opening example is

$$\dfrac{\dfrac{}{\cdot \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1}\ \text{T-Sort} \qquad \dfrac{\dfrac{}{\Gamma_1 \vdash P : \mathrm{Sort}\ 0}\ \text{T-Var} \qquad \dfrac{}{\Gamma_2 \vdash h : P}\ \text{T-Var}}{\Gamma_1 \vdash \lambda(h{:}P).\, h : P \to P}\ \text{T-Lam}}{\cdot \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}\ \text{T-Lam}$$

The algorithm builds the same derivation from the bottom up. Infer$(\cdot, \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h)$ makes the following calls:

1. InferSort$(\cdot, \mathrm{Sort}\ 0)$: Infer returns Sort 1, so the annotation is a type. (T-Sort)
2. Freshen leaves P unchanged, since the context is empty.
3. Infer$(\Gamma_1, \lambda(h{:}P).\, h)$, which makes three calls of its own:
   (a) InferSort(Γ_1, P): Infer looks up P and returns Sort 0. (T-Var)
   (b) Freshen leaves h unchanged.
   (c) Infer(Γ_2, h) looks up h and returns P. (T-Var)

   It returns $\Pi(h{:}P).\, P$. (inner T-Lam)

The outer call returns $\Pi(P{:}\mathrm{Sort}\ 0).\, \Pi(h{:}P).\, P$. (outer T-Lam)

Each call that returns a type corresponds to one node of the tree, and no call uses conversion. The result is α-equivalent to the stated type, because $P \to P$ abbreviates $\Pi(_{:}P).\, P$ and neither h nor _ occurs in the body.

A wrong proof. Suppose a user claims that $t = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, P$ proves $\Pi(P{:}\text{Sort } 0).\, P \to P$. The kernel calls $\textsc{Check}(\cdot, t, \Pi(P{:}\text{Sort } 0).\, P \to P)$, which first calls Infer. The calls are the same as above, except that the last one, $\textsc{Infer}(\Gamma_2, P)$, returns Sort 0. So Infer succeeds with

$$\textsc{Infer}(\cdot, t) = \Pi(P{:}\text{Sort } 0).\, \Pi(h{:}P).\, \text{Sort } 0.$$

The term t is well typed; it is just not a proof of the claimed proposition. Check then compares the two types, which are already normal, using the rules of Fig. 1:

$\cdot\,;\cdot \vdash \Pi(P{:}\text{Sort } 0).\, \Pi(h{:}P).\, \text{Sort } 0 =_\alpha \Pi(P{:}\text{Sort } 0).\, \Pi(_{:}P).\, P$	by α-Pi, needs
$\quad\cdot\,;\cdot \vdash \text{Sort } 0 =_\alpha \text{Sort } 0$	holds by α-Sort
$\quad P; P \vdash \Pi(h{:}P).\, \text{Sort } 0 =_\alpha \Pi(_{:}P).\, P$	by α-Pi, needs
$\quad\quad P; P \vdash P =_\alpha P$	holds by α-Bound (index 0)
$\quad\quad P, h; P, _ \vdash \text{Sort } 0 =_\alpha P$	no rule applies

so Check reports a type mismatch. The error is found only because the user stated the proposition to be proved. A kernel that only ran Infer would accept t and silently assign it a different type.

Conversion. Recall $Id = \lambda(X{:}\text{Sort } 0).\, X$ from §5. Our system has no definitions, so $Id\ P$ below is shorthand for the term $(\lambda(X{:}\text{Sort } 0).\, X)\ P$. Consider checking

$$t = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}Id\ P).\, h \quad \text{against} \quad \Pi(P{:}\text{Sort } 0).\, P \to P.$$

Infer first checks that the annotation $Id\ P$ is a type. $\textsc{InferSort}(\Gamma_1, Id\ P)$ reaches the application case of Infer: the function $\lambda(X{:}\text{Sort } 0).\, X$ has type $\Pi(X{:}\text{Sort } 0).\, \text{Sort } 0$, the argument P passes $\textsc{Check}(\Gamma_1, P, \text{Sort } 0)$, and the result is $\text{Sort } 0[X \mapsto P] = \text{Sort } 0$. The body h then has type $Id\ P$, so

$$\textsc{Infer}(\cdot, t) = \Pi(P{:}\text{Sort } 0).\, \Pi(h{:}Id\ P).\, Id\ P.$$

This is not α-equivalent to $\Pi(P{:}\text{Sort } 0).\, P \to P$. But Check normalizes both sides before comparing them, and $Id\ P \to_\beta P$, so the inferred type becomes $\Pi(P{:}\text{Sort } 0).\, \Pi(h{:}P).\, P$ and the comparison succeeds as in the first example. In the typing rules, this step is an application of T-Conv at the root of the derivation:

$$\frac{\cdot \vdash t : \Pi(P{:}\text{Sort } 0).\, Id\ P \to Id\ P \qquad \cdot \vdash \Pi(P{:}\text{Sort } 0).\, P \to P : \text{Sort } 0 \qquad \Pi(P{:}\text{Sort } 0).\, Id\ P \to Id\ P \equiv \Pi(P{:}\text{Sort } 0).\, P \to P}{\cdot \vdash t : \Pi(P{:}\text{Sort } 0).\, P \to P}\ \text{T-Conv}$$

The algorithm never decides where to apply T-Conv. As described in §8, conversion is hidden in the calls to Normalize, which run only where two types are compared.

An exercise. Prove $\Pi(P{:}\text{Sort } 0).\, \Pi(Q{:}\text{Sort } 0).\, P \to Q \to P$. One answer is

$$\lambda(P{:}\text{Sort } 0).\, \lambda(Q{:}\text{Sort } 0).\, \lambda(p{:}P).\, \lambda(q{:}Q).\, p.$$

The proof never uses q; this system allows hypotheses to go unused. Running Infer on it by hand, as in the first example, is a good check of one's understanding of the algorithm.

10 A PITFALL: VARIABLE CAPTURE IN CONTEXTS

It is tempting to drop the side condition $x \notin \text{dom}(\Gamma)$ and let a new assumption simply shadow an old one, as most programming languages do. Doing so makes the kernel unsound. Consider

$$t = \lambda(x{:}\text{Sort } 0).\, \lambda(y{:}x).\, \lambda(x{:}\text{Sort } 0).\, y.$$

The variable y has the *outer* x as its type, and the innermost λ ignores its argument. So the correct type of t is

$$\Pi(x{:}\text{Sort } 0).\, \Pi(y{:}x).\, \Pi(x'{:}\text{Sort } 0).\, x.$$

A checker without Freshen looks up y in the context $x : \text{Sort } 0, y : x, x : \text{Sort } 0$ and returns its type, the bare name x. At that point x refers to the inner binder, and the checker returns

$$\Pi(x{:}\text{Sort } 0).\, \Pi(y{:}x).\, \Pi(x{:}\text{Sort } 0).\, x,$$

which says "from any element of any type, produce an element of every type". Given $A : \text{Sort } 0$ and $a : A$, the term $t\ A\ a$ then has type $\Pi(B{:}\text{Sort } 0).\, B$, a proof of False. Such A and a exist even in the empty context: $A = \Pi(P{:}\text{Sort } 0).\, P \to P$ is itself a proposition (§7.4), and the identity function $a = \lambda(P{:}\text{Sort } 0).\, \lambda(h{:}P).\, h$ proves it.

The problem is the same as in substitution (§3): a name is moved into a scope where it means something else. It affects any representation that identifies variables by name, whether the context is a list or a hash map. There are three common remedies:

(1) Reject shadowing outright. This is the simplest option, but the reserved name _ used by $A \to B$ must be exempt and never added to the context.
(2) Rename on entry, as Freshen does.
(3) Represent bound variables by position (de Bruijn indices [6]) or use the *locally nameless* style, in which bound variables are indices and free variables carry unique identifiers. Lean 4 takes the latter approach.

A test suite should include t above and check that the kernel either rejects it or assigns it the correct type.

11 PROPERTIES

We state, without proof, the standard properties of this system. It is a Pure Type System: the Calculus of Constructions [2] with a hierarchy of predicative universes above its impredicative Sort 0, close to Luo's Extended Calculus of Constructions [3]. The first three properties below hold for every functional Pure Type System [5]. Strong normalization is much harder in the presence of impredicativity and needs Girard's method of reducibility candidates [2, 3, 7]. §12 revisits these properties after adding inductive types.

Theorem 3 (Uniqueness of types). *If $\Gamma \vdash t : A$ and $\Gamma \vdash t : B$, then $A \equiv B$.*

Theorem 4 (Type correctness). *If $\Gamma \vdash t : A$, then either $A = \text{Sort } u$ for some u, or $\Gamma \vdash A : \text{Sort } u$ for some u.*

Theorem 5 (Subject reduction). *If $\Gamma \vdash t : A$ and $t \to_\beta t'$, then $\Gamma \vdash t' : A$.*

Theorem 6 (Strong normalization). *If $\Gamma \vdash t : A$, then every sequence of β-reductions starting from t is finite.*

Together these give the two guarantees we want from a kernel. *Decidability*: Algorithm 1 terminates on every input, so whether a proof is correct can always be determined. *Consistency*: there is no closed term of type $\Pi(P{:}\text{Sort } 0).\, P$. By strong normalization and subject reduction, such a term would have a normal form of the same type. That normal form must be $\lambda(P{:}\text{Sort } 0).\, s$ with $P : \text{Sort } 0 \vdash s : P$ and s normal. A normal term is a universe, a Π, a λ, or a variable applied to arguments, and none of these can have type P in that context: the only variable available is P itself, whose type is Sort 0, not P.

12 INDUCTIVE TYPES: NATURAL NUMBERS AND EQUALITY

So far the only types are universes and Π types. This is enough for propositional logic, but not for mathematics: there are no numbers to reason about, and no way to state that two things are equal. Lean provides both through *inductive types*. Declarations such as

```
inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat

inductive Eq {A : Sort u} : A -> A -> Prop where
  | refl (a : A) : Eq a a
```

make the kernel add a type, its constructors, and a *recursor* that eliminates it, together with computation rules for the recursor. A general kernel derives all of these from the declaration. We first add the two types above by hand, which shows the ingredients, and then explain in §12.5 how they become data that a fixed kernel processes.

12.1 Constants and Definitions

Terms gain *constants* c:

$$t, A, B ::= \cdots \mid c$$

A *signature* Σ lists the constants. Each entry either declares a constant with its type, written $c : C$, or also gives it a body, written $c : C := t$; the latter are *definitions*. For example, Nat : Sort 1 below is a declaration, and add in §12.2 is a definition. Before an entry is added, its type is checked to be a type in the empty context, and its body, if any, is checked against that type, as Lean does for each `def`.

The signature is used in two ways. First, a constant has the type the signature gives it:

$$\frac{\vdash \Gamma \text{ ok} \qquad (c : C) \in \Sigma}{\Gamma \vdash c : C} \text{ (T-Const)}$$

where $(c : C) \in \Sigma$ covers both declarations and definitions. Second, a definition can be *unfolded*: the constant is replaced by its body. This computation step is called δ*-reduction*:

$$c \to_\delta t \qquad \text{if } (c : C := t) \in \Sigma.$$

For example, add 2 1 $\to_\delta$ $(\lambda(m{:}\text{Nat}).\, \lambda(n{:}\text{Nat}).\, \cdots)$ 2 1, after which β-reduction continues. A declared constant without a body, such as Nat or zero, never δ-reduces. Like β, the step may be applied to any subterm.

Constants are not variables: they are never bound, so $\text{FV}(c) = \emptyset$ and substitution leaves them unchanged. Reduction now consists of β, δ, and the ι-rules introduced below, and conversion (§5) is taken with respect to all three.

12.2 Natural Numbers

Fig. 4 lists the constants for Nat and Eq and their ι-rules. The recursor Nat.rec_u takes a *motive* M, which says what is to be built for each n, a value z for zero, and a step s that turns a value for n into a value for succ n. The ι-rules say that recursion unfolds along the constructors. As in Lean, where the recursor is `Nat.rec.{u}`, there is one recursor for each universe level; since we have no universe polymorphism, the level is a subscript.

The same recursor does two jobs, depending on the level.

Recursion. At level 1 with a constant motive, Nat.rec_1 defines functions by primitive recursion. Addition, defined by recursion on the second argument as in Lean, is the definition

$$\text{add} : \text{Nat} \to \text{Nat} \to \text{Nat} := \lambda(m{:}\text{Nat}).\, \lambda(n{:}\text{Nat}).\, \text{Nat.rec}_1\ M\ m\ s\ n,$$

Constant and type		Lean
Nat	: Sort 1	`Nat`
zero	: Nat	`Nat.zero`
succ	: Nat → Nat	`Nat.succ`
$\mathsf{Nat.rec}_u$	: $\Pi(M{:}\mathsf{Nat} \to \mathsf{Sort}\ u).\ M\ \mathsf{zero} \to$ $(\Pi(n{:}\mathsf{Nat}).\ M\ n \to M\ (\mathsf{succ}\ n)) \to \Pi(n{:}\mathsf{Nat}).\ M\ n$	`Nat.rec`
Eq_u	: $\Pi(A{:}\mathsf{Sort}\ u).\ A \to A \to \mathsf{Sort}\ 0$	`Eq`
refl_u	: $\Pi(A{:}\mathsf{Sort}\ u).\ \Pi(a{:}A).\ \mathsf{Eq}_u\ A\ a\ a$	`Eq.refl`
$\mathsf{Eq.ndrec}_{u,v}$	: $\Pi(A{:}\mathsf{Sort}\ u).\ \Pi(a{:}A).\ \Pi(M{:}A \to \mathsf{Sort}\ v).$ $M\ a \to \Pi(b{:}A).\ \mathsf{Eq}_u\ A\ a\ b \to M\ b$	`Eq.ndrec`

$$\mathsf{Nat.rec}_u\ M\ z\ s\ \mathsf{zero} \to_\iota z$$
$$\mathsf{Nat.rec}_u\ M\ z\ s\ (\mathsf{succ}\ n) \to_\iota s\ n\ (\mathsf{Nat.rec}_u\ M\ z\ s\ n)$$
$$\mathsf{Eq.ndrec}_{u,v}\ A\ a\ M\ m\ b\ (\mathsf{refl}_u\ A\ c) \to_\iota m$$

Fig. 4. Constants for natural numbers and equality (top) and their ι-rules (bottom).

with the constant motive $M = \lambda(_{:}\mathsf{Nat}).\ \mathsf{Nat}$ and the step $s = \lambda(k{:}\mathsf{Nat}).\ \lambda(r{:}\mathsf{Nat}).\ \mathsf{succ}\ r$. We write $m + n$ for add $m\ n$, and $0, 1, 2, \ldots$ for zero, succ zero, ... The kernel computes $2 + 1$ as

$$\begin{aligned} 2 + 1 &\to_{\delta\beta} \mathsf{Nat.rec}_1\ M\ 2\ s\ (\mathsf{succ}\ 0) \ \to_\iota\ s\ 0\ (\mathsf{Nat.rec}_1\ M\ 2\ s\ 0) \\ &\to_\beta \mathsf{succ}\ (\mathsf{Nat.rec}_1\ M\ 2\ s\ 0) \ \to_\iota\ \mathsf{succ}\ 2 = 3. \end{aligned}$$

Induction. At level 0, the motive $M : \mathsf{Nat} \to \mathsf{Sort}\ 0$ is a predicate on natural numbers, and the type of $\mathsf{Nat.rec}_0$ reads as the induction principle:

$$M\ \mathsf{zero} \to (\forall n.\ M\ n \to M\ (\mathsf{succ}\ n)) \to \forall n.\ M\ n.$$

Recursion and induction are the same constant at different levels. Here the choice of imax in §7.4 is essential. The conclusion $\Pi(n{:}\mathsf{Nat}).\ M\ n$ has type $\mathsf{Sort}\ \mathrm{imax}(1, 0) = \mathsf{Sort}\ 0$, so "$M\ n$ for every n" is a proposition although Nat lives in Sort 1. With max, it would live in Sort 1, and statements about numbers would not be propositions.

12.3 Equality

For $a, b : A$, the type $\mathsf{Eq}_u\ A\ a\ b$ is the proposition that a and b are equal; when A is Nat we write it $a = b$. Its only constructor refl_u proves $a = a$. The recursor $\mathsf{Eq.ndrec}_{u,v}$ is Leibniz's principle: if $a = b$, then whatever holds of a holds of b. It *transports* $m : M\ a$ along a proof of $a = b$ to $M\ b$, and its ι-rule says that transporting along refl does nothing. In a well-typed redex, b and c are both convertible to a, so the rule can ignore them. Lean's primitive recursor `Eq.rec` has a motive that may also depend on the proof of equality; Lean derives `Eq.ndrec` from it, and the non-dependent version is enough here.

In Lean, the type argument of Eq is *implicit*, written `{A}` above, and the elaborator fills it in. Our kernel sees only fully explicit terms, so we write every argument; filling them in is the job of the layer above the kernel (§13).

Definitional versus propositional equality. There are now two notions of equality. Definitional equality $\equiv$ is decided by the kernel by computation; propositional equality Eq is a type, whose proofs may require reasoning. The constructor refl connects them: by T-Conv, $\mathsf{refl}_u\ A\ a : \mathsf{Eq}_u\ A\ a\ b$

whenever $a \equiv b$. For example,

$$n + 0 \;\to_{\delta\beta}\; \mathsf{Nat.rec}_1\ M\ n\ s\ \mathsf{zero} \;\to_{\iota}\; n,$$

so $\mathsf{refl}_1\ \mathsf{Nat}\ n : n + 0 = n$, and Lean proves this by `rfl`. In contrast, $0 + n \to_{\delta\beta} \mathsf{Nat.rec}_1\ M\ 0\ s\ n$ is stuck: n is a variable, and the ι-rules fire only on zero and succ. So $0 + n = n$ is not provable by refl and needs induction.

12.4 Example: $0 + n = n$

We prove $\Pi(n{:}\mathsf{Nat}).\ 0 + n = n$ by induction, using $\mathsf{Nat.rec}_0$ with the motive $M_0 = \lambda(n{:}\mathsf{Nat}).\ 0 + n = n$.

- *Base.* We need $M_0\ 0$, which is convertible to $0 + 0 = 0$. Since $0 + 0$ computes to 0, the term $\mathsf{refl}_1\ \mathsf{Nat}\ 0$ works.
- *Step.* Given $k : \mathsf{Nat}$ and $ih : 0 + k = k$, we need $M_0\ (\mathsf{succ}\ k)$, that is, $0 + \mathsf{succ}\ k = \mathsf{succ}\ k$. By the second ι-rule, $0 + \mathsf{succ}\ k$ computes to $\mathsf{succ}\ (0 + k)$, so it suffices to prove $\mathsf{succ}\ (0 + k) = \mathsf{succ}\ k$. We start from $\mathsf{refl}_1\ \mathsf{Nat}\ (\mathsf{succ}\ (0 + k))$ and transport it along ih with the motive $N = \lambda(b{:}\mathsf{Nat}).\ \mathsf{succ}\ (0 + k) = \mathsf{succ}\ b$, which replaces the second $0 + k$ by k:

$$step = \lambda(k{:}\mathsf{Nat}).\ \lambda(ih{:}0 + k = k).\ \mathsf{Eq.ndrec}_{1,0}\ \mathsf{Nat}\ (0 + k)\ N\ (\mathsf{refl}_1\ \mathsf{Nat}\ (\mathsf{succ}\ (0 + k)))\ k\ ih.$$

The proof is

$$zero_add = \mathsf{Nat.rec}_0\ M_0\ (\mathsf{refl}_1\ \mathsf{Nat}\ 0)\ step\ :\ \Pi(n{:}\mathsf{Nat}).\ 0 + n = n.$$

The kernel checks it with Algorithm 1. Each argument is checked against the corresponding part of the type of $\mathsf{Nat.rec}_0$, instantiated with M_0, and every comparison succeeds after normalization. Writing such terms by hand is tedious, and finding the right motive is the hard part; tactics (§13) automate both.

12.5 Computation Rules as Data

Fig. 4 writes each ι-rule as a separate equation, which suggests that every new inductive type needs new reduction code in the kernel. It does not. A kernel stores the ι-rules as data in the signature, next to the recursor, and a single piece of code applies them for every inductive type.

Each recursor entry records the position of its *major premise*, the argument whose constructor decides which rule applies, and a list of *rules*, one per constructor. A rule names the constructor, the number of its *fields* (the arguments that the constructor stores, as opposed to the type parameters it merely carries), and a right-hand side *rhs*, an ordinary term. Each constructor entry records how many leading arguments are parameters. For our two recursors, the data is as follows; type annotations on λ are omitted.

Recursor	Major	Constructor	Params	Fields	*rhs*
$\mathsf{Nat.rec}_u$	4th	zero	0	0	$\lambda M\ z\ s.\ z$
		succ	0	1	$\lambda M\ z\ s\ n.\ s\ n\ (\mathsf{Nat.rec}_u\ M\ z\ s\ n)$
$\mathsf{Eq.ndrec}_{u,v}$	6th	refl_u	2	0	$\lambda A\ a\ M\ m\ b.\ m$

The *rhs* takes the arguments before the major premise, then the fields of the constructor. For example, $\mathsf{Nat.rec}_u\ M\ z\ s\ (\mathsf{succ}\ n)$ selects the second rule and becomes $(\lambda M\ z\ s\ n.\ s\ n\ (\mathsf{Nat.rec}_u\ M\ z\ s\ n))\ M\ z\ s\ n$, which β-reduces to the right-hand side of the second equation in Fig. 4.

Algorithm 2 extends Whnf of Algorithm 1 with two cases, one for δ and one for ι; Normalize is unchanged.

The new cases have the following properties.

Algorithm 2 WHNF with δ and ι. Lines 6–12 are new; the rest is as in Algorithm 1.

```
1:  function Whnf(t)
2:      loop
3:          write t as a spine h a_1 ··· a_n
4:          if h = λ(x:A). b and n ≥ 1 then                                  ▷ β
5:              t ← (b[x ↦ a_1]) a_2 ··· a_n
6:          else if h = c and (c : C := d) ∈ Σ then                          ▷ δ
7:              t ← d a_1 ··· a_n
8:          else if h = c is a recursor with major position p ≤ n then       ▷ ι
9:              k b_1 ··· b_j ← Whnf(a_p)
10:             if k is a constructor with a rule r of c, and k has q parameters then
11:                 t ← r.rhs a_1 ··· a_{p−1} b_{q+1} ··· b_j a_{p+1} ··· a_n
12:             else return t                                  ▷ stuck, e.g. on a variable
13:         else return t
```

- The ι case contains no reference to Nat or Eq. It reads the major position and the rules from the signature. These two cases are added to the kernel once; after that, each new inductive type adds rows of data, not code.
- The major premise is put in weak head normal form before the rules are consulted, because it may become a constructor only after computation, as in $2 + 1$ above. If it does not, for instance because it is a variable, the term is stuck and stays as it is; this is why $0 + n$ does not reduce.

12.6 General Inductive Types

In Lean, a user never writes the rows above. When it accepts an `inductive` declaration, the kernel generates the type former, the constructors, the recursor with its type, and the rules, all from the shapes of the constructors. For `Nat`, it generates exactly the entries of Fig. 4 and the table above. For `Eq`, it generates the dependent recursor `Eq.rec`, from which `Eq.ndrec` is then defined. So the kernel's rules stay fixed while the signature grows: one generic procedure turns each declaration into signature entries, and Algorithm 2 runs their rules.

The procedure must also reject some declarations. The declaration

```
inductive Bad where
  | mk : (Bad -> False) -> Bad
```

mentions `Bad` to the left of an arrow in the type of a constructor argument. Case analysis on it would let one define $f : \texttt{Bad} \to \texttt{False}$ by $f\ (\texttt{mk}\ g) = g\ (\texttt{mk}\ g)$. Then $f\ (\texttt{mk}\ f)$ is a closed term of type `False`, and it reduces to itself, so it has no normal form. The self-application is the same kind of self-reference as in Russell's paradox (§7.3). Lean therefore requires every constructor argument to mention the type being defined only *strictly positively*, that is, never to the left of an arrow.

12.7 Changes to the Kernel and Its Properties

INFER gains a case for constants, which looks up the type in Σ, and NORMALIZE keeps its definition from Algorithm 1, with WHNF replaced by Algorithm 2. CHECK is unchanged: it still normalizes both types and compares them up to α-equivalence.

The ι-rules do not overlap with each other or with β, so reduction remains confluent. Subject reduction, strong normalization, and consistency are known for the Calculus of Inductive Constructions [4], of which our system is a small fragment. The short consistency argument of §11 no longer suffices by itself, because normal forms now include stuck recursor applications; consistency

follows from strong normalization by a more careful analysis of normal forms. Lean's full theory, with definitional proof irrelevance and general inductive types, has subtler metatheory [1].

13 TACTICS: BUILDING PROOF TERMS INTERACTIVELY

Lean users rarely write terms like *zero_add* by hand. They write a tactic script instead:

```
theorem zero_add (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih]
```

Each tactic is a small program that builds part of the proof term. Tactics run outside the kernel. Their output is an ordinary term, which the kernel checks with Algorithm 1 like any other term. This section shows how the common tactics construct terms in our system.

13.1 Goals and Proof States

A *goal* $\langle \Gamma \vdash ?_m : T \rangle$ asks for a term of type T in the context Γ. The *hole* $?_m$ stands for the term still to be found; Lean calls it a *metavariable*. A *proof state* is a list of goals together with a *partial proof term*, a term that may contain holes.

To prove T, we start from the partial term $?_0$ and the single goal $\langle \cdot \vdash ?_0 : T \rangle$. A tactic works on the first goal $\langle \Gamma \vdash ?_m : T \rangle$. It *assigns* to $?_m$ a term that may contain fresh holes, and replaces the goal by one new goal for each fresh hole. When no goals remain, the partial term contains no holes, and the kernel checks it against T. A hole may sit under binders, as $?_1$ does in $\lambda(x{:}A).\, ?_1$, and the term that fills it may use those bound variables. That is why each goal carries its own context.

Each tactic must make sure that its assignment has type T in Γ, assuming each fresh hole is later filled with a term of the type its goal states. We write $\langle \Gamma \vdash ?_m : T \rangle \rightsquigarrow$ new goals, followed by the assignment.

13.2 Basic Tactics

`intro x`. If T normalizes to a Π type, the tactic moves its argument into the context:

$$\langle \Gamma \vdash ?_m : \Pi(y{:}A).\, B \rangle \;\rightsquigarrow\; \langle \Gamma, x : A \vdash ?_1 : B[y \mapsto x] \rangle, \qquad ?_m := \lambda(x{:}A).\, ?_1.$$

The assignment has the right type by T-Lam. The name x must not be in $\mathrm{dom}(\Gamma)$; otherwise the new assumption would shadow an existing one, which makes the kernel unsound (§10), so the tactic must rename it.

`exact t`. The tactic closes the goal with a given term:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \text{no goals}, \qquad ?_m := t,$$

after running $\textsc{Check}(\Gamma, t, T)$ itself. The kernel would reject an ill-typed t at the end anyway; checking early only gives a better error message.

`apply f`. If f has type $A_1 \to \cdots \to A_n \to B$ with $B \equiv T$, the tactic leaves the arguments as new goals:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_1 : A_1 \rangle, \ldots, \langle \Gamma \vdash ?_n : A_n \rangle, \qquad ?_m := f\ ?_1 \cdots ?_n.$$

The assignment has type B by n uses of T-App, and type T by T-Conv. When the type of f is a dependent Π, later A_i and B mention earlier arguments, and the tactic must find those arguments by matching B against T. Lean does this by *unification*, solving for metavariables. Our restricted version handles only non-dependent arrows.

`change T'`.. The tactic replaces the goal by a definitionally equal one:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_1 : T' \rangle, \qquad ?_m :=?_1,$$

provided T' is a type and $T' \equiv T$. It adds nothing to the term; it only changes how the goal is displayed, and T-Conv justifies it. `change` is needed because `rw` (below) finds subterms only up to α-equivalence: a subterm that appears only after computation must first be exposed with `change`, as in the example of §13.5.

`rfl`. If T normalizes to $\mathsf{Eq}_u\ A\ a\ b$ with $a \equiv b$, the tactic closes the goal:

$$\langle \Gamma \vdash ?_m : \mathsf{Eq}_u\ A\ a\ b \rangle \;\rightsquigarrow\; \text{no goals}, \qquad ?_m := \mathsf{refl}_u\ A\ a.$$

The assignment has type $\mathsf{Eq}_u\ A\ a\ a$, which is convertible to the goal. So `rfl` proves exactly the equations that hold by definitional equality (§12.3).

13.3 Induction

Let $n : \mathsf{Nat}$ be a variable in Γ, and let $\Gamma \vdash T : \mathsf{Sort}\ u$. The tactic `induction n` abstracts n in the goal to form the motive, and produces one goal per constructor:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_z : T[n \mapsto \mathsf{zero}] \rangle,\ \langle \Gamma, k : \mathsf{Nat}, ih : T[n \mapsto k] \vdash ?_s : T[n \mapsto \mathsf{succ}\ k] \rangle,$$
$$?_m := \mathsf{Nat.rec}_u\ (\lambda(z{:}\mathsf{Nat}).\, T[n \mapsto z])\ ?_z\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(ih{:}T[n \mapsto k]).\, ?_s)\ n.$$

With the motive $M = \lambda(z{:}\mathsf{Nat}).\, T[n \mapsto z]$, the type of $\mathsf{Nat.rec}_u\ M$ asks for $M\ \mathsf{zero}$ and $\Pi(k{:}\mathsf{Nat}).\, M\ k \to M\ (\mathsf{succ}\ k)$, and the result has type $M\ n$. These are convertible to the types of the two goals and to T, by β. The construction is correct only if the names are fresh, the level u is right, and no other hypothesis mentions n:

- The motive binds a fresh name z, not n: since n is already in Γ, writing $\lambda(n{:}\mathsf{Nat}).\, T$ would shadow it, which is the pitfall of §10. Likewise k and ih must be fresh.
- The level u of the motive comes from the type of the goal, found by InferSort. For a proposition it is 0, and $\mathsf{Nat.rec}_0$ is induction.
- If another hypothesis mentions n, for example $h : n = 2$, the new goals still contain h about the original n, and ih says nothing about it. The goals are then correct but may be unprovable. Lean's `induction` first moves such hypotheses into the goal (it *reverts* them), so that they are part of the motive. Our version requires that no other hypothesis mentions n.

13.4 Rewriting

Let $h : \mathsf{Eq}_u\ A\ a\ b$, and let the goal T be a proposition. The tactic `rw [h]` replaces a by b in the goal in four steps.

(1) *Abstract.* Choose a fresh x and let C be T with every occurrence of a replaced by x. Occurrences are found up to α-equivalence, not up to conversion, and an occurrence under a binder that binds a free variable of a is skipped, since there it denotes something else.
(2) *Check the motive.* Check that $\Gamma, x : A \vdash C : \mathsf{Sort}\ 0$. This can fail, as shown below.
(3) *Build the term.* The new goal is $C[x \mapsto b]$, the goal with a replaced by b:

$$\langle \Gamma \vdash ?_m : T \rangle \;\rightsquigarrow\; \langle \Gamma \vdash ?_1 : C[x \mapsto b] \rangle,$$
$$?_m := \mathsf{Eq.ndrec}_{u,0}\ A\ a\ (\lambda(x{:}A).\, C \to T)\ (\lambda(p{:}T).\, p)\ b\ h\ ?_1.$$

(4) *Try* `rfl`. Like Lean, close the new goal by `rfl` if possible.

To see that the assignment has type T, write $N = \lambda(x{:}A).\, C \to T$ for the motive. Since $C[x \mapsto a] = T$, the term $N\ a$ is convertible to $T \to T$, which the identity $\lambda(p{:}T).\, p$ inhabits. Transport along h then gives $N\ b$, which is convertible to $C[x \mapsto b] \to T$, and applying it to $?_1$ gives T. Lean builds a different term, `Eq.mpr (congrArg (fun x => C) h)`, from the same idea.

When the motive is ill typed. Abstracting a can break the goal. Let $\Gamma = n : \mathsf{Nat},\ h : n = 2,\ q : n = 0$, and suppose we run `rw [h]` on the goal $\mathsf{Eq}_0\ (n = 0)\ q\ q$, a statement about the proof q. Abstracting n gives $C = \mathsf{Eq}_0\ (x = 0)\ q\ q$, which is ill typed: q has type $n = 0$, not $x = 0$, and q itself cannot be abstracted. Step 2 fails, and Lean reports the same situation as "motive is not type correct". Without step 2, the tactic would produce a term that the kernel rejects.

13.5 Example: $0 + n = n$ with Tactics

In our system the proof of §12.4 becomes the script

```
intro n
induction n
rfl
change succ (0 + k) = succ k
rw [ih]
```

The proof state evolves as follows.

(1) Initially there is one goal, $\langle \cdot \vdash ?_0 : \Pi(n{:}\mathsf{Nat}).\, 0 + n = n \rangle$.

(2) `intro n` assigns $?_0 := \lambda(n{:}\mathsf{Nat}).\, ?_1$ and leaves $\langle n : \mathsf{Nat} \vdash ?_1 : 0 + n = n \rangle$.

(3) `induction n` assigns

$$?_1 := \mathsf{Nat.rec}_0\ (\lambda(z{:}\mathsf{Nat}).\, 0 + z = z)\ ?_2\ (\lambda(k{:}\mathsf{Nat}).\, \lambda(ih{:}0 + k = k).\, ?_3)\ n$$

and leaves two goals: $\langle n : \mathsf{Nat} \vdash ?_2 : 0 + 0 = 0 \rangle$ and $\langle n : \mathsf{Nat}, k : \mathsf{Nat}, ih : 0 + k = k \vdash ?_3 : 0 + \mathsf{succ}\ k = \mathsf{succ}\ k \rangle$.

(4) `rfl` closes the first goal with $?_2 := \mathsf{refl}_1\ \mathsf{Nat}\ (0 + 0)$, since $0 + 0 \equiv 0$.

(5) In the second goal, $0 + k$ does not occur syntactically, so `rw [ih]` would find nothing. `change` first exposes it, using $0 + \mathsf{succ}\ k \equiv \mathsf{succ}\ (0 + k)$: it assigns $?_3 := ?_4$ with the goal $?_4 : \mathsf{succ}\ (0 + k) = \mathsf{succ}\ k$. In Lean, rewriting with the lemma `Nat.add_succ`, which states $n + \mathsf{succ}\ m = \mathsf{succ}\ (n + m)$, produces the same goal.

(6) `rw [ih]` abstracts $0 + k$ to get $C = (\mathsf{succ}\ x = \mathsf{succ}\ k)$. With $T = (\mathsf{succ}\ (0 + k) = \mathsf{succ}\ k)$ for the current goal, it assigns

$$?_4 := \mathsf{Eq.ndrec}_{1,0}\ \mathsf{Nat}\ (0 + k)\ (\lambda(x{:}\mathsf{Nat}).\, C \to T)\ (\lambda(p{:}T).\, p)\ k\ ih\ ?_5.$$

The new goal $?_5 : \mathsf{succ}\ k = \mathsf{succ}\ k$ is closed at once by `rfl`, with $?_5 := \mathsf{refl}_1\ \mathsf{Nat}\ (\mathsf{succ}\ k)$.

No goals remain. Substituting the assignments into $?_0$ gives a closed term, and the kernel runs $\textsc{Check}(\cdot, t, \Pi(n{:}\mathsf{Nat}).\, 0 + n = n)$ on it. The term differs from the hand-written one of §12.4 in two places: it starts with $\lambda(n{:}\mathsf{Nat}).\ \ldots\ n$ because of `intro`, and its step uses the motive built by `rw`. Both are proofs of the same proposition, and the kernel does not care which one it receives.

13.6 Why Tactics Need Not Be Trusted

The kernel checks the final term, so tactics do not have to be correct for the system to be sound. A buggy tactic can fail, or it can produce an ill-typed term, which the kernel rejects; it cannot make the kernel accept a false theorem. The checks that tactics perform themselves, such as $\textsc{Check}$ in `exact` or the motive check in `rw`, exist only to report errors early and clearly. The trusted part of the system is therefore small: Algorithm 1 with the extended $\textsc{Whnf}$ of Algorithm 2, and the signature of §12.

Lean's tactic framework is far richer than the one sketched here. It has metavariables solved by unification, an elaborator that fills in implicit arguments and expands notation such as `0 + n`, an `induction` that reverts dependent hypotheses, and automation such as `simp`. All of this lives on the untrusted side: Lean's kernel, like ours, only ever sees complete terms [9].

REFERENCES

[1] M. Carneiro. *The Type Theory of Lean*. Master's thesis, Carnegie Mellon University, 2019.
[2] T. Coquand and G. Huet. The calculus of constructions. *Information and Computation*, 76(2–3), 1988.
[3] Z. Luo. *An Extended Calculus of Constructions*. PhD thesis, University of Edinburgh, 1990.
[4] B. Werner. *Une théorie des constructions inductives*. PhD thesis, Université Paris 7, 1994.
[5] H. P. Barendregt. Lambda calculi with types. In *Handbook of Logic in Computer Science*, 1992.
[6] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation. *Indagationes Mathematicae*, 1972.
[7] J.-Y. Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis, 1972.
[8] A. J. C. Hurkens. A simplification of Girard's paradox. *TLCA*, 1995.
[9] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE*, 2021.

A PROOF OF THEOREM 1

Both sides of Theorem 1 turn out to say the same thing: s and t have the same de Bruijn representation. We first define that representation, and then relate it to the algorithm (Lemma 7) and to the definition (Lemma 10).

Nameless terms. A *nameless term* is built like a term, except that binders carry no names and a bound variable is written as a number $\#i$:

$$d ::= \mathrm{Sort}\ u \mid \#i \mid x \mid \Pi\, d.\, d \mid \lambda\, d.\, d \mid d\ d$$

Free variables keep their names. For a list of names E, the nameless form $\lfloor t \rfloor_E$ of a term t that occurs under the binders E is

$$\lfloor \mathrm{Sort}\ u \rfloor_E = \mathrm{Sort}\ u \qquad \lfloor x \rfloor_E = \begin{cases} \#idx(E, x) & \text{if } x \in E, \\ x & \text{otherwise,} \end{cases}$$

$$\lfloor \Pi(x{:}A).\, B \rfloor_E = \Pi\, \lfloor A \rfloor_E.\, \lfloor B \rfloor_{E,x} \qquad \lfloor \lambda(x{:}A).\, t \rfloor_E = \lambda\, \lfloor A \rfloor_E.\, \lfloor t \rfloor_{E,x}$$

$$\lfloor t_1\ t_2 \rfloor_E = \lfloor t_1 \rfloor_E\ \lfloor t_2 \rfloor_E.$$

Equality of nameless terms is plain syntactic equality. Note that $\lfloor t \rfloor_E$ has the same outermost shape as t, except that a variable may become either a number or a name. Constants (§12.1) are left unchanged, like universes, and are compared by name; they add only trivial cases below.

Lemma 7 (The algorithm compares nameless forms). *For all E_1, E_2, s, and t: $E_1; E_2 \vdash s =_\alpha t$ is derivable if and only if $\lfloor s \rfloor_{E_1} = \lfloor t \rfloor_{E_2}$.*

Proof. By induction on s, for all t, E_1, and E_2. For each shape of s, exactly one rule of Fig. 1 can conclude the judgment, and its premises match the equations for $\lfloor \cdot \rfloor$.

- If $s = \mathrm{Sort}\ u$, both sides hold exactly when $t = \mathrm{Sort}\ u$, by α-Sort.
- If $s = x$, the nameless forms can be equal only if t is a variable y. If $x \in E_1$, then $\lfloor x \rfloor_{E_1} = \#idx(E_1, x)$, and equality holds exactly when $y \in E_2$ and $idx(E_2, y) = idx(E_1, x)$, which is the premise of α-Bound. If $x \notin E_1$, then $\lfloor x \rfloor_{E_1} = x$, and equality holds exactly when $y \notin E_2$ and $y = x$, which is the premise of α-Free. A number never equals a name, matching the absence of a rule for a variable that is bound on one side only.
- If $s = \Pi(x{:}A).\, B$, equality holds exactly when $t = \Pi(y{:}A').\, B'$ with $\lfloor A \rfloor_{E_1} = \lfloor A' \rfloor_{E_2}$ and $\lfloor B \rfloor_{E_1,x} = \lfloor B' \rfloor_{E_2,y}$. By the induction hypothesis, these are the premises of α-Pi. The cases λ and application are the same. □

The definition of $=_\alpha$ involves substitution, so we need two facts about how nameless forms behave under renaming. The first says that $\lfloor t \rfloor_E$ depends on E only through the free variables of t.

Lemma 8 (Agreement). *If $idx(E_1, v) = idx(E_2, v)$ for every $v \in \mathrm{FV}(t)$, with both sides possibly undefined, then $\lfloor t \rfloor_{E_1} = \lfloor t \rfloor_{E_2}$.*

Proof. By induction on t, for all E_1 and E_2. The variable case is the assumption. Under a binder x, the lists E_1, x and E_2, x again agree on the free variables of the body: x has position 0 in both, and any other free variable has its position increased by one in both, or stays undefined in both. □

The second says that renaming a binder does not change the nameless form. We use that substituting a variable for a variable preserves the size of a term, even when binders are renamed; this is a routine induction.

Lemma 9 (Renaming). *Let $y \neq x$, $y \notin \mathrm{FV}(B)$, and $x, y \notin E'$. Then $\lfloor B \rfloor_{E,x,E'} = \lfloor B[x \mapsto y] \rfloor_{E,y,E'}$ for every list E.*

Proof. By induction on the size of B, for all E and E'. Write $L_x = E, x, E'$ and $L_y = E, y, E'$; they differ only in one entry, at the same position.

- If $B = x$, then $B[x \mapsto y] = y$. Since $x, y \notin E'$, both $idx(L_x, x)$ and $idx(L_y, y)$ equal the length of E'.
- If $B = v \neq x$, then $v \neq y$ because $y \notin \mathrm{FV}(B)$, and $B[x \mapsto y] = v$. The position of v is the same in L_x and L_y, or undefined in both.
- Universes and applications are immediate from the definitions and the induction hypothesis.
- Let $B = \Pi(w{:}C).\, D$; the case of λ is the same. The annotation C is handled by the induction hypothesis in each case below, so we consider only the body. There are three cases, following the definition of substitution.
  - $w = x$. Then $B[x \mapsto y] = \Pi(x{:}C[x \mapsto y]).\, D$, and we need $\lfloor D \rfloor_{L_x,x} = \lfloor D \rfloor_{L_y,x}$. We apply Lemma 8. The variable x has position 0 in both lists. Any other $v \in \mathrm{FV}(D)$ differs from y, since $y \notin \mathrm{FV}(B)$, so its position is the same in both lists, or undefined in both.
  - $w \notin \{x, y\}$. Then $B[x \mapsto y] = \Pi(w{:}C[x \mapsto y]).\, D[x \mapsto y]$. The induction hypothesis for D, with E' extended by w, gives $\lfloor D \rfloor_{L_x,w} = \lfloor D[x \mapsto y] \rfloor_{L_y,w}$.
  - $w = y$. The binder would capture y, so substitution renames it to a fresh $z \notin \mathrm{FV}(D) \cup \{x, y\}$: $B[x \mapsto y] = \Pi(z{:}C[x \mapsto y]).\, (D[y \mapsto z])[x \mapsto y]$. The induction hypothesis for D, renaming its last binder y to z, gives $\lfloor D \rfloor_{L_x,y} = \lfloor D[y \mapsto z] \rfloor_{L_x,z}$. The induction hypothesis for $D[y \mapsto z]$, which has the size of D and no free y, with E' extended by z, gives $\lfloor D[y \mapsto z] \rfloor_{L_x,z} = \lfloor (D[y \mapsto z])[x \mapsto y] \rfloor_{L_y,z}$. □

Lemma 10 (The definition compares nameless forms). *If $s =_\alpha t$ in the sense of Definition 3, then $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for every E. Conversely, if $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for some E, then $s =_\alpha t$.*

Proof. For the first part, let $s \sim t$ mean that $\lfloor s \rfloor_E = \lfloor t \rfloor_E$ for every E. This relation is an equivalence, and it is a congruence, because $\lfloor \cdot \rfloor$ is defined constructor by constructor. It contains the defining equation of Definition 3: if $y \notin \mathrm{FV}(B)$, then $\lfloor \Pi(x{:}A).\, B \rfloor_E = \Pi\, \lfloor A \rfloor_E.\, \lfloor B \rfloor_{E,x}$ and $\lfloor \Pi(y{:}A).\, B[x \mapsto y] \rfloor_E = \Pi\, \lfloor A \rfloor_E.\, \lfloor B[x \mapsto y] \rfloor_{E,y}$, and the bodies are equal by Lemma 9 with E' empty (the case $y = x$ is trivial). Since $=_\alpha$ is the smallest congruence containing that equation, $s =_\alpha t$ implies $s \sim t$.

For the converse, we use induction on the size of s, for all t and E.

- If s is a variable, so is t. If both nameless forms are $\#i$, both variables are the entry at position i of E, so $s = t$. If both are names, then $s = t$ directly.
- Universes and applications follow from the induction hypothesis and the congruence of $=_\alpha$.

- Let $s = \Pi(x{:}A).\, B$, so that $t = \Pi(y{:}A').\, B'$ with $\lfloor A \rfloor_E = \lfloor A' \rfloor_E$ and $\lfloor B \rfloor_{E,x} = \lfloor B' \rfloor_{E,y}$. The induction hypothesis gives $A =_\alpha A'$. For the bodies, choose $z \notin \mathrm{FV}(B) \cup \mathrm{FV}(B') \cup \{x, y\}$. Lemma 9 with E' empty gives $\lfloor B \rfloor_{E,x} = \lfloor B[x \mapsto z] \rfloor_{E,z}$ and $\lfloor B' \rfloor_{E,y} = \lfloor B'[y \mapsto z] \rfloor_{E,z}$, so $\lfloor B[x \mapsto z] \rfloor_{E,z} = \lfloor B'[y \mapsto z] \rfloor_{E,z}$. Since $B[x \mapsto z]$ has the size of B, the induction hypothesis gives $B[x \mapsto z] =_\alpha B'[y \mapsto z]$. Then

  $$\Pi(x{:}A).\, B \;=_\alpha\; \Pi(z{:}A).\, B[x \mapsto z] \;=_\alpha\; \Pi(z{:}A').\, B'[y \mapsto z] \;=_\alpha\; \Pi(y{:}A').\, B',$$

  by the defining equation, by congruence, and by the defining equation read backwards. The case of λ is the same. □

Proof of Theorem 1. If $s =_\alpha t$, then $\lfloor s \rfloor_\cdot = \lfloor t \rfloor_\cdot$ by Lemma 10, and $\cdot ; \cdot \vdash s =_\alpha t$ by Lemma 7. Conversely, if $\cdot ; \cdot \vdash s =_\alpha t$, then $\lfloor s \rfloor_\cdot = \lfloor t \rfloor_\cdot$ by Lemma 7, and $s =_\alpha t$ by Lemma 10. □

B PROOF OF THEOREM 2

The precise statement. Theorem 2 leaves implicit which proposition variables are in scope. For the proof we make them explicit and write $\Theta; \Delta \vdash \varphi$, where Θ is a list of distinct proposition variables and every proposition variable free in Δ or φ belongs to Θ. The rules of Fig. 3 then read

$$\frac{\varphi \in \Delta}{\Theta; \Delta \vdash \varphi} \qquad \frac{\Theta; \Delta, \varphi \vdash \psi}{\Theta; \Delta \vdash \varphi \to \psi} \qquad \frac{\Theta; \Delta \vdash \varphi \to \psi \qquad \Theta; \Delta \vdash \varphi}{\Theta; \Delta \vdash \psi} \qquad \frac{\Theta, P; \Delta \vdash \varphi \qquad P \notin \Theta}{\Theta; \Delta \vdash \forall P.\, \varphi}$$

$$\frac{\Theta; \Delta \vdash \forall P.\, \varphi \qquad \mathrm{FV}(\psi) \subseteq \Theta}{\Theta; \Delta \vdash \varphi[P \mapsto \psi]}$$

The side condition $P \notin \mathrm{FV}(\Delta)$ of Fig. 3 is now implied by $P \notin \Theta$. Formulas are taken up to α-equivalence, and $\varphi^* \to \psi^*$ stands for $\Pi(x{:}\varphi^*).\, \psi^*$ with x fresh.

The context Δ^* of §7.1 lists the proposition variables first and the hypotheses after them. The proof has to extend contexts on the right, so it is convenient to allow any order.

Definition 5 (Representation). *A context* Γ represents $(\Theta; \Delta)$ *if* $\vdash \Gamma$ ok *and* Γ *consists of one assumption* $P : \mathrm{Sort}\ 0$ *for each* $P \in \Theta$ *and one assumption* $h : \chi^*$ *for each* $\chi \in \Delta$*, in some order.*

Theorem 11 (Theorem 2, precise form). *If* $\Theta; \Delta \vdash \varphi$ *and* Γ *represents* $(\Theta; \Delta)$*, then* $\Gamma \vdash t : \varphi^*$ *for some term* t*.*

The context Δ^* represents $(\Theta; \Delta)$ when Θ lists the free proposition variables; it is well formed by Lemma 12 below and C-Ext. So Theorem 2 is the special case $\Gamma = \Delta^*$.

Facts about the translation. Translation commutes with substitution: $(\varphi[P \mapsto \psi])^* = \varphi^*[P \mapsto \psi^*]$. This is a routine induction on φ.

Lemma 12 (Formulas are propositions). *If* $\vdash \Gamma$ ok *and* $(P : \mathrm{Sort}\ 0) \in \Gamma$ *for every* $P \in \mathrm{FV}(\varphi)$*, then* $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$.

Proof. By induction on φ. For P, use T-Var. For $\varphi \to \psi$, the induction hypothesis gives $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$, so $\Gamma, x : \varphi^*$ is well formed by C-Ext for a fresh x. The induction hypothesis for ψ in that context gives $\mathrm{Sort}\ 0$, and T-Pi gives $\mathrm{Sort}\ \mathrm{imax}(0, 0) = \mathrm{Sort}\ 0$. For $\forall Q.\, \chi$, rename Q so that $Q \notin \mathrm{dom}(\Gamma)$. T-Sort gives $\Gamma \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1$, so $\Gamma, Q : \mathrm{Sort}\ 0$ is well formed, and T-Pi with the induction hypothesis for χ gives $\mathrm{Sort}\ \mathrm{imax}(1, 0) = \mathrm{Sort}\ 0$. □

The last case is where impredicativity is used. With max in place of imax, $(\forall Q.\, \chi)^*$ would live in $\mathrm{Sort}\ 1$, and the $\forall$-elim case below would hold only for formulas ψ without $\forall$.

Proof of Theorem 11. By induction on the derivation of $\Theta; \Delta \vdash \varphi$, for all Γ at once. Each case builds the typing derivation shown in the right column of Fig. 3.

- *Hypothesis.* Γ contains $h : \varphi^*$, so $\Gamma \vdash h : \varphi^*$ by T-Var.
- $\to$*-intro.* By Lemma 12, $\Gamma \vdash \varphi^* : \mathrm{Sort}\ 0$. For a fresh h, the context $\Gamma, h : \varphi^*$ is well formed and represents $(\Theta; \Delta, \varphi)$. The induction hypothesis gives $\Gamma, h : \varphi^* \vdash t : \psi^*$, and T-Lam gives $\Gamma \vdash \lambda(h{:}\varphi^*).\, t : \Pi(h{:}\varphi^*).\, \psi^*$. This type is $\varphi^* \to \psi^*$, since the free variables of ψ^* are proposition variables and h is not one of them.
- $\to$*-elim.* The induction hypotheses give $\Gamma \vdash f : \Pi(x{:}\varphi^*).\, \psi^*$ with $x \notin \mathrm{FV}(\psi^*)$ and $\Gamma \vdash a : \varphi^*$. T-App gives $\Gamma \vdash f\ a : \psi^*[x \mapsto a] = \psi^*$.
- $\forall$*-intro.* Renaming P if necessary, assume $P \notin \mathrm{dom}(\Gamma)$; a routine induction shows that renaming a variable of Θ to a fresh one preserves derivations. By T-Sort and C-Ext, $\Gamma, P : \mathrm{Sort}\ 0$ is well formed, and it represents $(\Theta, P; \Delta)$. The induction hypothesis gives $\Gamma, P : \mathrm{Sort}\ 0 \vdash t : \varphi^*$, and T-Lam gives $\Gamma \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, t : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^*$.
- $\forall$*-elim.* The induction hypothesis gives $\Gamma \vdash f : \Pi(P{:}\mathrm{Sort}\ 0).\, \varphi^*$, and Lemma 12 gives $\Gamma \vdash \psi^* : \mathrm{Sort}\ 0$. T-App gives $\Gamma \vdash f\ \psi^* : \varphi^*[P \mapsto \psi^*] = (\varphi[P \mapsto \psi])^*$. □